POLIMI GRADUATE SCHOOL OF MANAGEMENT

Netflix: The NoSQL Transition

Author(s): **Nur Alam - 11182370**

Alexandre Benjamin Cambier - 10797708

Simone Cobuccio - 11162496

Giancarlo Ferrara - 11174662

Davide Tissino - 10940644

Academic Year: 2025-2026

Contents

Contents **i**

1 Why a Non Relational Database? **1**
1.1 Introduction . . . 1
1.1.1 Scenario . . . 1
1.1.2 Role of Data . . . 1
1.1.3 The Problem . . . 2

2 Non Relational Approach to Movies **3**
2.1 Our Project . . . 3
2.2 Our Dataset . . . 4
2.2.1 Dataset Structure . . . 4
2.2.2 Justification of Choice . . . 5
2.2.3 Dataset Preview . . . 5
2.3 Preliminary Queries . . . 6
2.3.1 Data Wrangling . . . 6
2.3.2 CRUD Operations . . . 8
2.4 Exploratory Queries . . . 14
2.4.1 Foundational Queries . . . 14
2.4.2 Intermediate Queries . . . 17
2.4.3 Advanced Queries . . . 21

3 Conclusion **30**
3.1 Relevant Takeaways . . . 30
3.2 Final Remarks . . . 31

1 | Why a Non Relational Database?

1.1. Introduction

1.1.1. Scenario

Netflix is an American subscription video on-demand streaming service, among the industry leaders, that serves approximately 200 million Monthly Active Users; specifically Netflix manages a massive catalog that includes both movies and TV shows that were either acquired or originally developed by the platform itself. Therefore, the core business challenge involves the ingestion, organization, and retrieval of content metadata for thousands of titles.

Traditionally, movies and TV shows metadata include information as titles, members of the cast, production year, duration, genre, etc...; this information was normally stored in Relational Database Management Systems (DBMS), which naturally rely on rigid, tabular schemas. However, as the metadata becomes more complex (e.g., varying durations, massive cast lists, and multiple genre tags), these traditional systems struggle with the flexibility required for rapid updates and scaling.

1.1.2. Role of Data

In this scenario, data is not a mere inventory of information but rather a strategic asset that provides measurable business value:

- **Operational Efficiency**: Managing semi-structured data allows for the storage of diverse metadata without the constant need for schema migrations.
- **Strategic Insights**: Analyzing content trends through data allows the platform to make informed decisions regarding content acquisition and regional investments.
- **User Experience Optimization**: Efficient data retrieval supports the underlying

recommendation engine and search functionalities that drive user retention.

1.1.3. The Problem

As previously mentioned, as movie metadata becomes more complex, traditional systems can struggle with flexibility. This project explores the transition from relational models to MongoDB, a document-oriented database. By moving to a Non-Relational approach, we can handle unstructured data more efficiently and perform deeper analyses. Throughout this study, we will demonstrate how migrating to MongoDB simplifies the way we store, query, and gain insights from the Netflix dataset.

2 | Non Relational Approach to Movies

2.1. Our Project

As the streaming industry continues to evolve, data management plays a crucial role in optimizing user experience, managing global catalogs, and supporting strategic decision-making. Our project focuses on migrating structured movie data from a CSV-based relational system into a scalable and flexible NoSQL database using MongoDB.

The motivation behind this transition stems from the need to:

- **Enhance Data Structure**: Traditional relational databases require rigid schemas, making it challenging to integrate new metadata like diverse genres or cast details. By introducing *subdocuments* in MongoDB, we aim to improve data retrieval speed and organizational efficiency.
- **Accommodate Diverse Data**: Netflix datasets often contain heterogeneous information, including movie descriptions, user ratings, and production details. A NoSQL approach allows for seamless integration of these different data types without the constraints of fixed tables.
- **Improve Analytical Capabilities**: With a more dynamic database structure, our system enables analysts to perform CRUD operations (Create, Read, Update, Delete) more efficiently while maintaining the integrity of the movie catalog.
- **Generate Key Insights**: By designing optimized queries, our aim is to extract meaningful insights related to content trends, production years, and global popularity, helping to better understand the vast Netflix library.

2.2. Our Dataset

To carry out this project, we identified a "**Netflix Movies and TV Shows**" dataset which was publicly available on Kaggle[1]. The dataset provides a comprehensive list of 8.807 instances of movies and TV shows available on the Netflix platform as of October 2021. The dataset includes relevant features together with metadata information, thus allowing for the following:

- **Content Diversity**: The dataset encompasses both feature movies and episodic TV shows across Netflix's global markets.
- **Metadata Richness**: It includes textual descriptions, categorical ratings (e.g., TV-MA, PG-13), and temporal data such as the date added to the platform.
- **Categorization**: Titles are mapped to multiple genres, which in a relational model would require complex many-to-many join tables, but here can be stored as flexible arrays.

2.2.1. Dataset Structure

Specifically, each instance in the chosen dataset follows the structure outlined in the following Table:

Table 2.1: Netflix Dataset Attribute Specifications

Attribute	Type	Description
`_id`	ObjectId	Internally generated unique identifier for the document.
`show_id`	String	Unique alphanumeric identifier for each title.
`type`	String	Categorization of the entry (e.g., "Movie" or "TV Show").
`title`	String	The official name of the content piece.
`director`	String	Primary director(s) of the title.
`cast`	String	List of featured actors involved in the production.
`country`	String	Geographical origin of the production.
`date_added`	String	The timestamp indicating when the title was added to the Netflix platform.

[1]Link to the Dataset on Kaggle: `https://www.kaggle.com/datasets/shivamb/netflix-shows`

Table 2.1: Netflix Dataset Attribute Specifications (Continued)

Attribute	Type	Description
`release_year`	Integer	The original year of premiere or broadcast.
`rating`	String	Target audience rating (e.g., PG-13, TV-MA).
`duration`	String	Quantitative length (minutes for movies, seasons for TV shows).
`listed_in`	String	Categories and genres associated with the title.
`description`	String	A brief narrative summary of the content.

2.2.2. Justification of Choice

Overall, the dataset we identified provides for the following:

- **Semi-Structured Nature:** Fields such as `cast` and `listed_in` contain multi-valued strings that are better represented as arrays in a NoSQL environment rather than through complex associative tables in a relational model.
- **Schema Flexibility:** The difference in `duration` for "Movies" (measured in minutes) and "TV Shows" (measured in seasons) demonstrates MongoDB's ability to handle heterogeneous attributes within a single collection.
- **Analytical Potential:** The variety of temporal (`release_year`), categorical (`rating`, `type`) and geographical (`country`) data allows for the implementation of complex aggregation pipelines.

2.2.3. Dataset Preview

To illustrate the document-oriented nature of the implementation, a preview of the initial records is provided below, in tabular format. This representation highlights how multi-valued attributes are stored as native arrays, a key advantage of the NoSQL approach.

Title	**Type**	**...**	**Cast (Selection)**	**Duration**	**...**	**Listed In**
Breaking Bad	TV Show	...	Bryan Cranston, Aaron Paul	5 Seasons	...	Crime TV Shows, TV Dramas
House of Cards	TV Show	...	Kevin Spacey, Robin Wright	6 Seasons	...	TV Dramas, TV Thrillers
[...]						
Grown Ups	Movie	...	Adam Sandler, Kevin James	103 min	...	Comedies
Jaws	Movie	...	Roy Scheider, Robert Shaw	124 min	...	Action, Adventure, Classic

Table 2.2: Sample Data Preview from the Netflix Dataset

2.3. Preliminary Queries

2.3.1. Data Wrangling

Before performing relevant queries on the dataset, we must execute data wrangling, that is the process of cleaning or transforming the data from its raw CSV state into a refined NoSQL format.

1. **Standardization of Date**: The raw `date_added` field is stored as a string (e.g., "September 25, 2021"). Therefore, converting this to a BSON `ISODate` object is necessary for chronological sorting and time-based aggregations. The transformation can be achieved through the following query:

```
db.Movies.find({
    date_added: { $type: "string", $ne: "" }
}).forEach(doc => {
    db.Movies.updateOne(
        { _id: doc._id },
        { $set: { date_added: new Date(doc.date_added.trim()) } }
    );
});
```

The following images highlight the before and after `date_added` field conversion, taking one specific TV show as a sample to showcase the change:

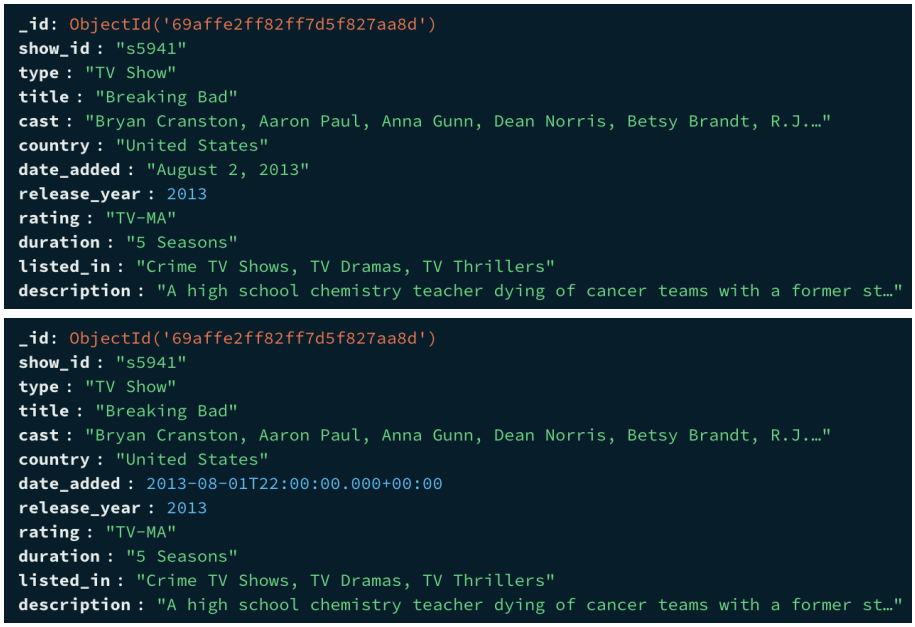

Figure 2.1: Instance preview before (top) and after (bottom) date conversion.

2| Non Relational Approach to Movies

2. **Handling Sparsity**: In the raw dataset, missing information for attributes like `director` or `country` was represented as empty strings (`""`). To improve data consistency, we executed a dynamic `updateMany` operation. Rather than hard-coding each attribute, the script extracts all field names from a sample document and iteratively replaces empty strings with an explicit "**N/A**" value.

```
// Get the field names from the first document
const fields = Object.keys(db.Movies.findOne());

// Loop through each field and update empty strings to "N/A"
fields.forEach(field => {
  db.Movies.updateMany(
    { [field]: "" },
    { $set: { [field]: "N/A" } });})
```

The following images highlight that the `country` field has been updated to `"N/A"` rather than being an empty string.

```
_id: ObjectId('69b0201bf82ff7d5f8281b8f')
show_id : "s8434"
type : "Movie"
title : "The Naked Gun 2 1/2: The Smell of Fear"
director : "David Zucker"
cast : "Leslie Nielsen, Priscilla Presley, George Kennedy, O.J. Simpson, Rober…"
country : ""
date_added : 2019-12-31T23:00:00.000+00:00
release_year : 1991
rating : "PG-13"
duration : "85 min"
listed_in : "Comedies"
description : "Bumbling cop Frank Drebin is out to foil the big boys in the energy in…"
```

```
_id: ObjectId('69b0201bf82ff7d5f8281b8f')
show_id : "s8434"
type : "Movie"
title : "The Naked Gun 2 1/2: The Smell of Fear"
director : "David Zucker"
cast : "Leslie Nielsen, Priscilla Presley, George Kennedy, O.J. Simpson, Rober…"
country : "N/A"
date_added : 2019-12-31T23:00:00.000+00:00
release_year : 1991
rating : "PG-13"
duration : "85 min"
listed_in : "Comedies"
description : "Bumbling cop Frank Drebin is out to foil the big boys in the energy in…"
```

Figure 2.2: Individual instance before (top) and after (bottom) N/A standardization.

2.3.2. CRUD Operations

In accordance with the project guidelines, we executed a series of structural operations to manage the NoSQL database.

1. **Inserting a New Movie (Create):** We manually added a new movie, the 2009 comedy "*The Hangover*", which was not present in the original dataset. It is critical to note that the inserted document strictly adheres to our post-wrangling schema, with`date_added` being cast as an `ISODate`.

```
db.Movies.insertOne({
    show_id: "s8808",
    type: "Movie",
    title: "The Hangover",
    director: "Todd Phillips",
    cast: "Bradley Cooper, Ed Helms, Zach Galifianakis, Justin Bartha, Ken Jeong",
    country: "United States",
    date_added: new Date("2026-03-10T00:00:00Z"),
    release_year: 2009,
    rating: "R",
    duration: "100 min",
    listed_in: "Comedies",
    description: "Three buddies wake up from a bachelor party in Las Vegas, with ..."
});
```

The **Create** query successfully creates a new document within our dataset: in the top image, we have "*No Results*" if searching for the specific title, while in the bottom image we have a positive match if running the query *after* creating the new movie document.

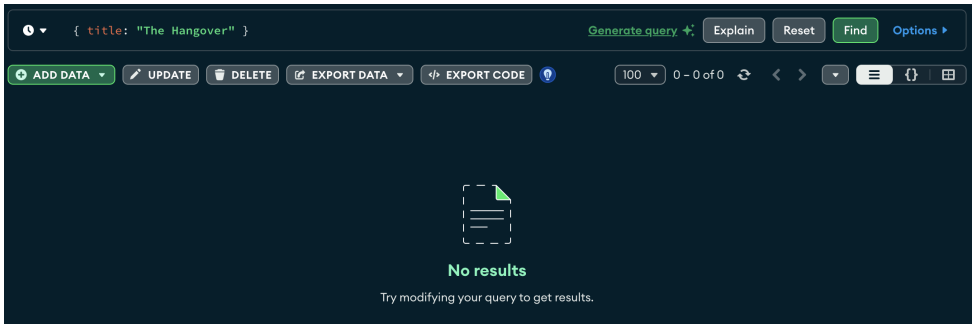

Figure 2.3: Individual instance before instance creation.

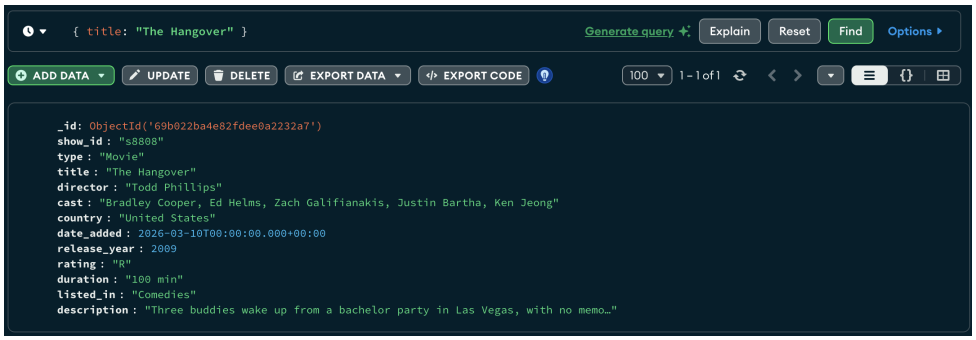

Figure 2.3: Individual instance after instance creation (Continued).

2. **Array Tokenization (Update):** In the raw format, fields `cast` and `listed_in` were stored as comma-separated strings; therefore, there is the need for a structural update to optimize the schema for NoSQL analytical queries. To leverage array-specific indexing and prepare the data for the `$unwind` aggregation operator, we executed an `updateMany` pipeline to tokenize these strings into native BSON arrays.

```
// Update cast field to array
db.Movies.updateMany(
    { cast: { $type: "string", $ne: "N/A" } },
    [{ $set: { cast: { $split: ["$cast", ", "] } } }]
);

// Update the listed_in field to array
db.Movies.updateMany(
    { listed_in: { $type: "string", $ne: "N/A" } },
    [{ $set: { listed_in: { $split: ["$listed_in", ", "] } } }]
);
```

The images highlight the change from string to array for `cast` and `listed_in` fields.

Figure 2.4: Sample instance before conversion to array.

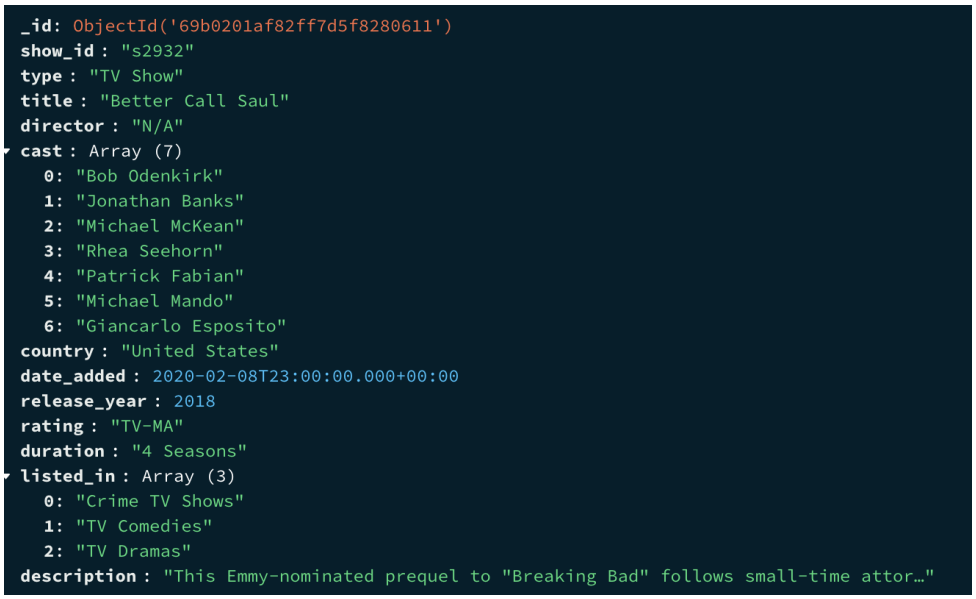

Figure 2.4: Sample instance after conversion to array (Continued).

3. **Insertion of Distinct Records (Create):** To contrast the single-document insertion and demonstrate bulk processing capabilities, we used the `insertMany` operation to simultaneously add multiple distinct TV-Show instances to the catalog. To simulate a targeted content acquisition update, we selected two distinct shows released after the dataset's 2021 cutoff: a sports docuseries and a global fantasy drama. These records are ingested strictly adhering to our optimized NoSQL schema, with multi-valued fields (`cast` and `listed_in`) natively structured as BSON arrays and `date_added` cast as an `ISODate`.

```
db.Movies.insertMany([
    // Insert the first show
    {
        show_id: "s8809",
        type: "TV Show",
        title: "Starting 5",
        director: "N/A",
        cast: [
            "LeBron James",
            "Anthony Edwards",
            "Jimmy Butler",
            "Domantas Sabonis",
            "Jayson Tatum"
        ],
```

```
        country: "United States",
        date_added: new Date("2024-10-09T00:00:00Z"),
        release_year: 2024,
        rating: "TV-MA",
        duration: "2 Seasons",
        listed_in: ["Docuseries", "Sports TV"],
        description: "A behind-the-scenes look at the NBA season through ..."
    },
    // Insert the second show
    {
        show_id: "s8810",
        type: "TV Show",
        title: "Wednesday",
        director: "Tim Burton",
        cast: [
            "Jenna Ortega",
            "Gwendoline Christie",
            "Riki Lindhome",
            "Christina Ricci",
            "Catherine Zeta-Jones"
        ],
        country: "United States",
        date_added: new Date("2022-11-23T00:00:00Z"),
        release_year: 2022,
        rating: "TV-14",
        duration: "2 Seasons",
        listed_in: ["TV Dramas", "TV Mysteries", "TV Sci-Fi & Fantasy"],
        description: "Smart, sarcastic and a little dead inside, Wednesday Addams ..."
    }
]);
```

As highlighted for the previous Create query, while the top image returns no match for the two specific TV-Shows titles, after running the query we actually have a positive match.

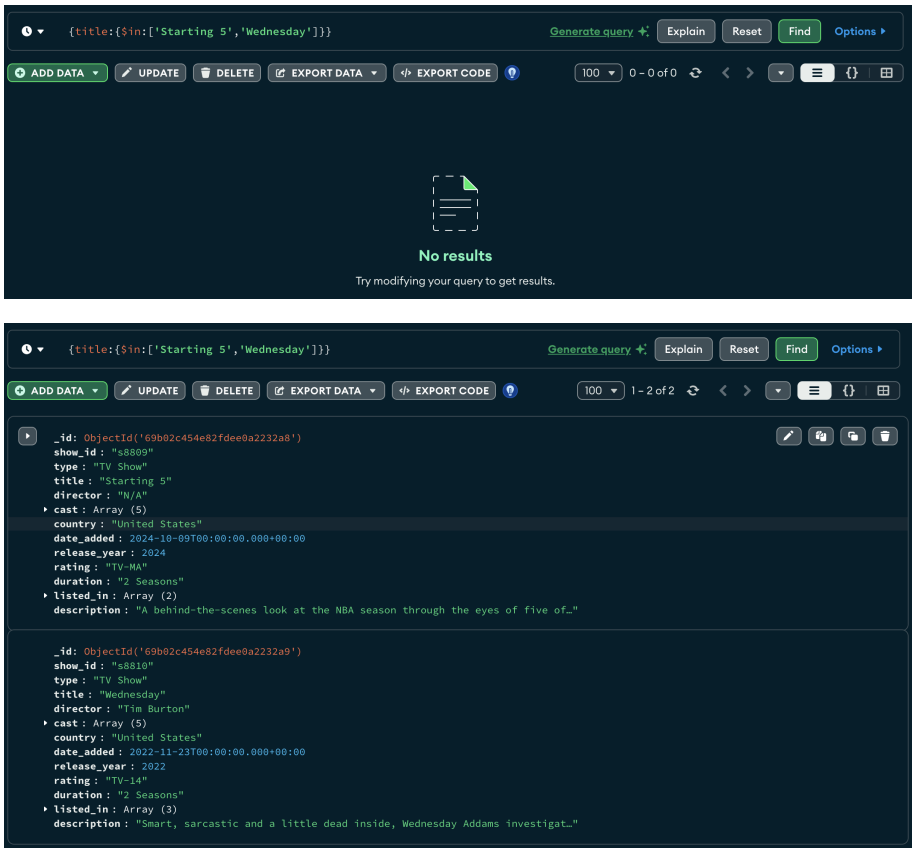

Figure 2.5: Before (top) and after (bottom) distinct instances creation.

4. **Archiving Legacy Content (Delete):** To pursue a Delete operation, we executed a `deleteMany` statement that resulted in the bulk removal of records. In the real-world streaming industry, databases are regularly pruned to remove legacy content with low viewer engagement or to clear out anomalous or outdated records. This specific query utilizes the `$or` logical operator combined with the `$lte` (less than or equal to) operator to permanently delete any instance released in or before 1950, as well as any records showing a `date_added` prior to 2001.

```
db.Movies.deleteMany({
    $or: [
        { release_year: { $lte: 1950 } },
        { date_added: { $lte: new Date("2000-12-31T23:59:59.000Z") } }
    ]
});
```

To provide a clear representation of the Delete operation, we first searched for the total number of instances that would be included in our filter using the `countDocuments` operator; this resulted in 16 movies and shows which were either released in or before

1950 or which have a `date_added` prior to 2001. After the `deleteMany` operation, the number of documents has been cut to zero.

```
> db.Movies.countDocuments({
  $or: [
  { release_year: { $lte: 1950 } },
  { date_added: { $lte: new Date("2000-12-31T23:59:59.000Z") } }
  ]
  });
< 16
> db.Movies.deleteMany({
  $or: [
  { release_year: { $lte: 1950 } },
  { date_added: { $lte: new Date("2000-12-31T23:59:59.000Z") } }
  ]
  });
< {
    acknowledged: true,
    deletedCount: 16
  }
> db.Movies.countDocuments({
  $or: [
  { release_year: { $lte: 1950 } },
  { date_added: { $lte: new Date("2000-12-31T23:59:59.000Z") } }
  ]
  });
< 0
```

Figure 2.6: Deletion of 16 records according to the chosen filter.

5. **Modifying an Array (Update):** For our final CRUD operation, we decided to modify existing multi-valued attributes without overwriting the entire field. Because we previously structured the `cast` field as a native array, we relied on the `$push` operator rather than the standard `$set` operator. This allowed us to append a new element, a group member, to the `cast` list of the TV-Show "Better Call Saul" at the end of the array, while preserving the other existing cast members.

```
db.Movies.updateOne(
    { title: "Better Call Saul" },
    // Add group member Simone Cobuccio to the cast
    { $push: { cast: "Simone Cobuccio" } }
);
```

```
> db.Movies.find(
      { title: "Better Call Saul" },
      { _id: 0, title: 1, cast: 1 }
  )
< {
    title: 'Better Call Saul',
    cast: [
      'Bob Odenkirk',
      'Jonathan Banks',
      'Michael McKean',
      'Rhea Seehorn',
      'Patrick Fabian',
      'Michael Mando',
      'Giancarlo Esposito'
    ]
  }
```

Before `$push`

```
> db.Movies.find(
      { title: "Better Call Saul" },
      { _id: 0, title: 1, cast: 1 }
  )
< {
    title: 'Better Call Saul',
    cast: [
      'Bob Odenkirk',
      'Jonathan Banks',
      'Michael McKean',
      'Rhea Seehorn',
      'Patrick Fabian',
      'Michael Mando',
      'Giancarlo Esposito',
      'Simone Cobuccio'
    ]
  }
```

After `$push`

Figure 2.7: Projection of the `cast` array before and after the update operation.

2.4. Exploratory Queries

With the Netflix dataset fully wrangled and the NoSQL schema optimized through the CRUD operations, we can now move onto a more "exploratory" phase to extract relevant business insights. In this section, we present a series of 10 analytical queries ranging from foundational filtering to more advanced aggregations to showcase how to query over temporal, multi-valued, and categorical data.

2.4.1. Foundational Queries

The first tier of Exploratory Queries focuses on basic document retrieval. Specifically, the queries showcase MongoDB's querying capabilities, targeted filtering and sorting mechanisms, and comprehensively simulate the essential operational requests that would be handled by a streaming platform's backend to validate catalog integrity.

1. **Recent TV-Shows Additions:** The first foundational query identifies the total number of TV-Shows that were added to the Netflix catalog during the calendar year 2021. We relied on the `countDocuments` operator to actually count all the instances, amounting to 496 documents. To optimize query performance, we relied on a range query with the `$gte` (greater than or equal to) and `$lt` (less than)

operators against the `ISODate` format, rather than dynamically extracting the year dynamically, to ensure that the query remains index-friendly.

```
db.Movies.countDocuments({
    "type": "TV Show",
    "date_added": {
        "$gte": ISODate("2021-01-01T00:00:00Z"),
        "$lt": ISODate("2022-01-01T00:00:00Z")
    }
});
```

The console output for the `countDocuments` operator shows a total of 496 TV-Shows that were added to the Netflix catalog in 2021.

```
> db.Movies.countDocuments(
      {
          "type": "TV Show",
          "date_added": {
              "$gte": ISODate("2021-01-01T00:00:00Z"),
              "$lt": ISODate("2022-01-01T00:00:00Z")
          }
      },
      { _id: 0, title: 1, date_added: 1, release_year: 1 }
  )
< 496
```

Figure 2.8: Count the number of TV-Shows added in 2021.

2. **Kid-Friendly Content:** This query identifies family-friendly feature films, specifically targeting movies rated "TV-Y" or "TV-Y7" with a runtime strictly under 60 minutes. Before executing this analysis, a preliminary operation was required; in the raw dataset, the `duration` field for movies was stored as a string appended with "min" (e.g., "60 min"). To enable the use of mathematical operators like `$lt` (less than), we executed an update pipeline using `$split` and `$toInt` to transform these strings into native integers. We then applied the `$in` operator to filter the categorical `rating` field, sorted the output according to ascending `duration` and eventually sliced it to only showcase the three "shortest" results for a concise output.

```
db.Movies.find(
    {
        type: "Movie",
```

```
            rating: { $in: ["TV-Y", "TV-Y7"] },
            duration: { $lt: 60 }
        },
        { _id: 0, title: 1, rating: 1, duration: 1 }
    ).sort({"duration": 1}).limit(3)
```

Additionally, using the `countDocuments` argument instead of `find` allowed us to assess that the overall number of movies satisfying our kid-friendly filter is 119, as highlighted in the right console output.

```
> db.Movies.find(
      {
          type: "Movie",
          rating: { $in: ["TV-Y", "TV-Y7"] },
          duration: { $lt: 60 }
      },
      { _id: 0, title: 1, rating: 1, duration: 1 }
  ).sort({"duration": 1}).limit(3)
< {
    title: 'Silent',
    rating: 'TV-Y',
    duration: 3
  }
  {
    title: 'Calico Critters: A Town of Dreams',
    rating: 'TV-Y',
    duration: 11
  }
  {
    title: "Calico Critters: Everyone's Big Dream Flying in the Sky",
    rating: 'TV-Y',
    duration: 11
  }
```

First three instances in ascending order

```
> db.Movies.countDocuments(
      {
          type: "Movie",
          rating: { $in: ["TV-Y", "TV-Y7"] },
          duration: { $lt: 60 }
      },
      { _id: 0, title: 1, rating: 1, duration: 1 }
  )
< 119
```

Total number of instances

Figure 2.9: Projection of kid-friendly movies lasting less than 60 minutes.

3. **Director Portfolio:** This query performs a targeted lookup to extract the complete movie portfolio of filmmaker Martin Scorsese. Specifically, it performs an exact string match on the categorical `director` field; subsequently, to enhance readability, we wrapped the standard MongoDB `find` operation within a custom JavaScript execution block. By converting the resulting output to an array using `.toArray()`, we then iterated through the retrieved documents to print a clean list of titles directly to the console.

```
// Find all movies by Martin Scorsese and convert to an array
const results = db.Movies.find(
  { type: "Movie", director: "Martin Scorsese" },
  { title: 1, _id: 0 }
).toArray();
```

```
// Iterate through the output set, print the portfolio to the console
print("Director: Martin Scorsese");
results.forEach(doc => {
  print("  - " + doc.title);
});
```

The previous iteration and custom execution allowed us to achieve the following clean and easily-readable output, rather than the longer document list which we would have obtained otherwise.

```
< Director: Martin Scorsese
<   - Shutter Island
<   - No Direction Home: Bob Dylan
<   - Hugo
<   - The Irishman
<   - Rolling Thunder Revue: A Bob Dylan Story by Martin Scorsese
<   - Alice Doesn't Live Here Anymore
<   - Gangs of New York
<   - GoodFellas
<   - Mean Streets
<   - Raging Bull
<   - The Departed
<   - Who's That Knocking at My Door?
```

Figure 2.10: Netflix movies catalog from director Martin Scorsese.

2.4.2. Intermediate Queries

Having previously optimized the schema by tokenizing comma-separated strings into arrays, this second tier of queries demonstrates the performance benefits of direct array querying and iteration to generate formatted, user-ready outputs.

1. **Actor Filmography:** This query extracts the complete filmography of a specific actor (Samuel L. Jackson) across both Movies and TV-Shows. Because the `cast` field was previously transformed into a native array during the wrangling phase, we directly queried it using an exact string match. This avoids the performance overhead of using regular expressions (`$regex`) to scan text. The results are sorted according to reverse chronological `release_year`. To format the output, we initialized a counter variable (`i`) and iterated through the result set using a `forEach` loop, printing a numbered, clean and readable list directly to the console.

```
// Initialize counter and print the header for the actor's filmography
let i = 1;
```

2| Non Relational Approach to Movies

```
print("Filmography: Samuel L. Jackson");

// Query the array directly, sort chronologically, and print formatted results
db.Movies.find(
  { "cast": "Samuel L. Jackson" },
  { "type": 1, "title": 1, "release_year": 1, "_id": 0 }
).sort({ "release_year": -1 }).forEach(movie => {
  print(`${i++}. ${movie.title}, Type: ${movie.type},
        Release Year: ${movie.release_year}`);
});
```

```
Filmography: Samuel L. Jackson
1. Amend: The Fight for America, Type: TV Show, Release Year: 2021
2. Death to 2020, Type: Movie, Release Year: 2020
3. Unicorn Store, Type: Movie, Release Year: 2019
4. The Incredibles 2, Type: Movie, Release Year: 2018
5. I Am Not Your Negro, Type: Movie, Release Year: 2017
6. The Hateful Eight: Extended Version, Type: TV Show, Release Year: 2015
7. The Hateful Eight, Type: Movie, Release Year: 2015
8. Barely Lethal, Type: Movie, Release Year: 2015
9. Turbo, Type: Movie, Release Year: 2013
10. Django Unchained, Type: Movie, Release Year: 2012
11. The Other Guys, Type: Movie, Release Year: 2010
12. Black Snake Moan, Type: Movie, Release Year: 2007
13. Freedomland, Type: Movie, Release Year: 2006
14. XXX: State of the Union, Type: Movie, Release Year: 2005
15. Coach Carter, Type: Movie, Release Year: 2005
16. S.W.A.T., Type: Movie, Release Year: 2003
17. XXx, Type: Movie, Release Year: 2002
18. Deep Blue Sea, Type: Movie, Release Year: 1999
19. Jackie Brown, Type: Movie, Release Year: 1997
20. 187, Type: Movie, Release Year: 1997
21. Pulp Fiction, Type: Movie, Release Year: 1994
22. National Lampoon's Loaded Weapon 1, Type: Movie, Release Year: 1993
23. Patriot Games, Type: Movie, Release Year: 1992
24. Do the Right Thing, Type: Movie, Release Year: 1989
```

Figure 2.11: Netflix movies and TV-shows catalog from actor Samuel L. Jackson.

2. **Top Content-Producing Countries:** We leveraged MongoDB's aggregation framework to identify the five most prolific countries on the Netflix platform. The pipeline executes in four distinct stages. First, a `$match` stage cleans the data stream by filtering out any documents where the `country` field is `"N/A"`. Then, the `$group` stage aggregates the remaining documents by their exact `country`, thus capturing both primary production nations and other international co-production partnerships, and

then calculates the total volume using the `$sum` operator. Finally, the pipeline applies a `$sort` operator to order the aggregated totals in descending order, followed by a `$limit` stage to restrict the output to the top-five results.

```
db.Movies.aggregate([
  // Filter out documents where country is N/A
  {
    $match: {
      country: { $nin: ["N/A"] }
    }
  },

  // Group by country and calculate the total count
  {
    $group: {
      _id: "$country",
      totalTitles: { $sum: 1 }
    }
  },

  // Sort in descending order
  {
    $sort: {
      totalTitles: -1
    }
  },

  // Restrict the final output to the top 5 entities
  {
    $limit: 5
  }
]);
```

```
< {
    _id: 'United States',
    totalTitles: 2807
  }
  {
    _id: 'India',
    totalTitles: 972
  }
  {
    _id: 'United Kingdom',
    totalTitles: 419
  }
  {
    _id: 'Japan',
    totalTitles: 245
  }
  {
    _id: 'South Korea',
    totalTitles: 199
  }
```

Figure 2.12: Top-5 most prolific countries on Netflix.

3. **Most-Cast Actors:** This query identifies the top 10 most frequently cast actors across the entire Netflix catalog. Because the `cast` field was restructured into a native array during the CRUD operations, we directly applied the `$unwind` operator to deconstruct the array itself. We then used `$group` to aggregate the total appearances per-actor and a `$match` stage to explicitly filter out any "N/A" value. After sorting and limiting the results, the aggregation cursor is converted to an array and iterated over, to dynamically print a clean, numbered leaderboard to the console.

```
db.Movies.aggregate([
  // Deconstruct the native cast array into individual documents
  { $unwind: "$cast" },

  // Group by actor name and calculate total appearances
  {
    $group: {
      _id: "$cast",
      totalAppearances: { $sum: 1 }
    } },

  // Filter out invalid data artifacts
```

```
    { $match: { _id: { $nin: ["N/A"] } } },

    // Sort in descending order and keep the top 10
    { $sort: { totalAppearances: -1 } },
    { $limit: 10 }
  ]).toArray().forEach((actor, index) => {
    print((index + 1) + ". " + actor._id + ": " + actor.totalAppearances);
  });
```

```
1. Anupam Kher: 43
2. Shah Rukh Khan: 35
3. Julie Tejwani: 33
4. Takahiro Sakurai: 32
5. Naseeruddin Shah: 32
6. Rupa Bhimani: 31
7. Om Puri: 30
8. Akshay Kumar: 30
9. Yuki Kaji: 29
10. Paresh Rawal: 28
```

Figure 2.13: Top-10 most-cast actors on Netflix.

2.4.3. Advanced Queries

The final tier of our exploratory analysis shifts towards data profiling, data manipulation and advanced aggregation pipelines. We leveraged operators such as `$facet` and conditional grouping to simulate more computation-heavy workloads typical of a streaming-platform's backend.

1. **Content Distribution Ratio:** This query analyzes the production distribution between Movies and TV-Shows on a per-country basis. We first tokenized and then unwound the comma-separated string `country` into distinct documents. Then, we used conditional aggregation (`$cond`) within a `$group` stage to sum the volume of Movies and TV Shows. After filtering out countries lacking representation in both categories (to prevent divide-by-zero errors), we calculated the production ratio of Movies to TV-Shows. Finally, the `$facet` stage is used to process the data in parallel, isolating and ranking the top-3 and bottom-3 countries with the highest and lowest Movie-to-TV-Show ratio, respectively.

2| Non Relational Approach to Movies

```
db.Movies.aggregate([
  // Tokenize comma-separated countries and unwind the array
  { $addFields: { country: { $split: ["$country", ", "] } } },
  { $unwind: "$country" },

  // Group by individual country and conditionally count Movies vs TV Shows
  {$group: {
      _id: "$country",
      Movies: { $sum: { $cond: [{ $eq: ["$type", "Movie"] }, 1, 0] } },
      TVShows: { $sum: { $cond: [{ $eq: ["$type", "TV Show"] }, 1, 0] } } }},

  // Filter out countries missing either category to avoid division by zero
  {$match: {
      Movies: { $gt: 0 },
      TVShows: { $gt: 0 } }},

  // Calculate the ratio of Movies to TV Shows, rounded to 2 decimal places
  {$addFields: {
      ratio: { $round: [{ $divide: ["$Movies", "$TVShows"] }, 2] }
    }},
  { $sort: { ratio: -1 } },

  // Process the sorted stream in parallel to extract highest and lowest ratios
  {$facet: {
      top3Highest: [
        { $limit: 3 },
        { $group: { _id: null, docs: { $push: "$$ROOT" } } },
        { $unwind: { path: "$docs", includeArrayIndex: "docs.rank" } },
        { $replaceRoot: { newRoot: "$docs" } },
        { $addFields: { rank: { $add: ["$rank", 1] } } } ],
      top3Lowest: [
        { $sort: { ratio: 1 } },
        { $limit: 3 },
        { $group: { _id: null, docs: { $push: "$$ROOT" } } },
        { $unwind: { path: "$docs", includeArrayIndex: "docs.rank" } },
        { $replaceRoot: { newRoot: "$docs" } },
        { $addFields: { rank: { $add: ["$rank", 1] } } } ]
    } } ]);
```

```
< {
    top3Highest: [
      {
        _id: 'United Arab Emirates',
        Movies: 36,
        TVShows: 1,
        ratio: 36,
        rank: 1
      },
      {
        _id: 'Philippines',
        Movies: 80,
        TVShows: 3,
        ratio: 26.67,
        rank: 2
      },
      {
        _id: 'Indonesia',
        Movies: 86,
        TVShows: 4,
        ratio: 21.5,
        rank: 3
      }
    ],
    top3Lowest: [
```

Top-3 countries with the highest movie-to-TV-show ratio

```
    ],
    top3Lowest: [
      {
        _id: 'Taiwan',
        Movies: 19,
        TVShows: 70,
        ratio: 0.27,
        rank: 1
      },
      {
        _id: 'South Korea',
        Movies: 61,
        TVShows: 170,
        ratio: 0.36,
        rank: 2
      },
      {
        _id: 'Ukraine',
        Movies: 1,
        TVShows: 2,
        ratio: 0.5,
        rank: 3
      }
    ]
  }
```

Bottom-3 countries with the lowest movie-to-TV-show ratio

Figure 2.14: Movies to TV-Show ratios per-country.

2. **Monthly Content Influx:** This query performs a temporal analysis to identify Netflix's busiest content-loading periods by aggregating titles based on the month they were added. We first used a `$match` stage to filter exclusively for documents where `date_added` is correctly cast as a native array. The `$group` stage then uses the `$month` date extraction operator to pull the numerical month from the timestamp, aggregating the total volume via the `$sum` operator. To enhance the interpretability of the results for relevant stakeholders, we implemented a `$switch` control flow statement within the `$addFields` stage to translate the numerical month identifiers (1-12) into categorical strings. Then, after projecting the relevant fields and sorting in descending order, we used a `print` statement to generate a clean output header, convert the resulting set to an array, and process it via a loop to print a ranked leaderboard directly to the console.

2| Non Relational Approach to Movies

```
print("Monthly Content Influx:");
db.Movies.aggregate([
  // Filter out missing or invalid dates
  { $match: { date_added: { $type: "date" } } },

  // Extract the numerical month and aggregate the total titles
  { $group: {
      _id: { $month: "$date_added" },
      totalAdded: { $sum: 1 } } },

  // Translate the integer into month
  { $addFields: {
      monthName: {
        $switch: {
          branches: [
            { case: { $eq: ["$_id", 1] }, then: "January" },
            { case: { $eq: ["$_id", 2] }, then: "February" },
            { case: { $eq: ["$_id", 3] }, then: "March" },
            { case: { $eq: ["$_id", 4] }, then: "April" },
            { case: { $eq: ["$_id", 5] }, then: "May" },
            { case: { $eq: ["$_id", 6] }, then: "June" },
            { case: { $eq: ["$_id", 7] }, then: "July" },
            { case: { $eq: ["$_id", 8] }, then: "August" },
            { case: { $eq: ["$_id", 9] }, then: "September" },
            { case: { $eq: ["$_id", 10] }, then: "October" },
            { case: { $eq: ["$_id", 11] }, then: "November" },
            { case: { $eq: ["$_id", 12] }, then: "December" }
          ],
          default: "Unknown"
        } } } },

  // Retain only the month name and count
  { $project: { _id: 0, monthName: 1, totalAdded: 1 } },

  // Sort from busiest month to slowest
  { $sort: { totalAdded: -1 } }

]).toArray().forEach((month, index) => {
  print((index + 1) + ". " + month.monthName +
```

```
" - " + month.totalAdded + " titles"); });
```

```
Monthly Content Influx:
1. December - 982 titles
2. June - 858 titles
3. September - 823 titles
4. July - 776 titles
5. August - 774 titles
6. March - 753 titles
7. October - 749 titles
8. April - 725 titles
9. February - 610 titles
10. November - 604 titles
11. May - 586 titles
12. January - 544 titles
```

Figure 2.15: Busiest content-loading months, sorted in descending order by uploads.

3. **Global Actor Collaborations:** For the penultimate advanced query, we diverge from targeting specific individuals towards a comprehensive analysis of the entire dataset. The objective is to identify the most frequent acting duos on the Netflix platform. Because the `cast` field is maintained as a native array, we utilize a highly complex `$project` stage featuring nested `$reduce`, `$map`, and `$filter` operators; this logic identifies unique actor pairings for every single title. To prevent duplicate permutations (e.g., counting Actor A & Actor B separately from Actor B & Actor A) and self-pairing, the pipeline enforces a strict alphabetical inequality operator (`$gt`). These unique pairs are then unwound, globally grouped, and aggregated to reveal the most frequent collaborators in the catalog, with the final cursor formatted into a readable leaderboard.

```
// Print the header for the console output
print("Top 10 Global Actor Collaborations:");

db.Movies.aggregate([
  // Filter out instances with missing cast data
  { $match: { cast: { $ne: "N/A" } } },

  // Only keep relevant fields
  { $project: { title: 1, cast: 1 } },

  // Pairing logic to generate unique actor duos per title
```

```
  { $project: {
      pairs: {
        $reduce: {
          input: "$cast",
          initialValue: [],
          in: {
            $concatArrays: [
              "$$value",
              {
                $map: {
                  input: {
                    $filter: {
                      input: "$cast",
                      as: "a",
                      // Prevents duplicates and self-pairs
                      cond: { $gt: ["$$a", "$$this"] }
                    }  },
                  as: "partner",
                  in: { actor1: "$$this", actor2: "$$partner" }
                } } ] } } } } },

  // Deconstruct the pairs, group globally and count collaborations
  { $unwind: "$pairs" },
  { $group: {
      _id: { actor1: "$pairs.actor1", actor2: "$pairs.actor2" },
      collaborations: { $sum: 1 } } },

  // Sort the highest collaborations descending and keep the top 10
  { $sort: { collaborations: -1 } },
  { $limit: 10 }
]).toArray().forEach((pair, index) => {
  // Format the output into a clean string
  print((index + 1) + ". " + pair._id.actor1 + " & " + pair._id.actor2 +
  ": " + pair.collaborations + " collaboration(s)");
});
```

```
Top 10 Global Actor Collaborations:
1. Julie Tejwani & Rupa Bhimani: 31 collaboration(s)
2. Julie Tejwani & Rajesh Kava: 24 collaboration(s)
3. Rajesh Kava & Rupa Bhimani: 22 collaboration(s)
4. Jigna Bhardwaj & Julie Tejwani: 21 collaboration(s)
5. Jigna Bhardwaj & Rupa Bhimani: 20 collaboration(s)
6. Jigna Bhardwaj & Rajesh Kava: 20 collaboration(s)
7. Julie Tejwani & Vatsal Dubey: 18 collaboration(s)
8. Rupa Bhimani & Vatsal Dubey: 18 collaboration(s)
9. Jigna Bhardwaj & Vatsal Dubey: 18 collaboration(s)
10. Rajesh Kava & Vatsal Dubey: 17 collaboration(s)
```

Figure 2.16: Top-10 global actors collaborations.

4. **Average Movie Duration by Rating:** This final query calculates the average runtime of Movies grouped by their target `rating`, subsequently ranking the results. To ensure computational stability against potentially uncleaned string fields (e.g., `duration` expressed as "90 min"), the pipeline relies on the `$addFields` stage, using a `$cond` operator to dynamically verify and cast data types to native integers at runtime. After calculating the mean duration via the `$group` and `$avg` operators, we used the `$setWindowFields` operator to process the grouped partitions in memory to subsequently assign a rank based on the sorted average runtime. Finally, a `$project` stage utilizes `$concat` and `$toString` to assemble a fully formatted string, which is then iterated and printed to the console.

```
// Print the header for the console output
print("Average Movie Duration by Rating Leaderboard:");

db.Movies.aggregate([
  // Filter for valid movies and exclude missing categorical data
  { $match: {
      type: "Movie",
      duration: { $nin: [null, "N/A", ""] },
      rating: { $nin: [null, "N/A", ""] } } },

  // Ensure duration is cast as an integer
  { $addFields: {
      durationInt: {
```

```
          $cond: [
            { $eq: [{ $type: "$duration" }, "string"] },
            { $toInt: { $arrayElemAt: [{ $split: ["$duration", " "] }, 0] } },
            { $toInt: "$duration" } ] } } },

    // Group by rating and calculate the mean duration
    { $group: {
        _id: "$rating",
        averageDuration: { $avg: "$durationInt" } } },

    // Clean the output, round the average to one decimal place
    { $project: {
        _id: 0,
        rating: "$_id",
        averageDuration: { $round: ["$averageDuration", 1] } } },

    // Generate a strict ranking according to duration
    { $setWindowFields: {
        sortBy: { averageDuration: -1 },
        output: {
          rank: { $documentNumber: {} } } } },

    // Render the final formatted string in the console
    { $project: {
        formattedString: {
          $concat: [
            { $toString: "$rank" },
            ". Rating: ",
            "$rating",
            ", Average Duration: ",
            { $toString: "$averageDuration" },
            " min(s)" ] } } }
  ]).forEach(doc => print(doc.formattedString));
```

```
Average Movie Duration by Rating Leaderboard:
1. Rating: NC-17, Average Duration: 125 min(s)
2. Rating: TV-14, Average Duration: 110.6 min(s)
3. Rating: PG-13, Average Duration: 108.3 min(s)
4. Rating: R, Average Duration: 106.7 min(s)
5. Rating: UR, Average Duration: 106.3 min(s)
6. Rating: PG, Average Duration: 98.3 min(s)
7. Rating: TV-MA, Average Duration: 95.9 min(s)
8. Rating: TV-PG, Average Duration: 95.3 min(s)
9. Rating: NR, Average Duration: 94.5 min(s)
10. Rating: G, Average Duration: 90.3 min(s)
11. Rating: TV-G, Average Duration: 79.7 min(s)
12. Rating: TV-Y7-FV, Average Duration: 68.4 min(s)
13. Rating: TV-Y7, Average Duration: 66.3 min(s)
14. Rating: TV-Y, Average Duration: 48.1 min(s)
```

Figure 2.17: Ranking of different ratings according to their average duration.

3 | Conclusion

3.1. Relevant Takeaways

To conclude, the comprehensive analyses we pursued allowed us to transform a raw dataset of 8,807 titles across Movies and TV-Shows into actionable and relevant insights. By executing a progressive series of queries, starting from foundational filtering to more advanced aggregations, we extracted several strategic insights addressing Netflix's catalog composition, regional investment strategies, and acquisition seasonality.

Through the analyses, we identified some primary insights:

- **Global Production Volume and Regional Focus:** While the United States predictably dominates the platform in raw production volume (2,807 titles) [Figure 2.12], India emerged as the second ranked country with 972 titles. This geographical focus cascades directly into talent analytics: the top 10 most-casted actors globally, led by Anupam Kher with 43 appearances [Figure 2.13], and the most frequent collaborating duos [Figure 2.16] are primarily composed of Indian talent, highlighting Netflix's aggressive expansion into Bollywood and regional Indian cinema.

- **Regional Content Strategies:** The advanced Movie and TV-Show ratio analysis [Figure 2.14] revealed that different global markets serve entirely different content needs for the platform. Asian markets such as South Korea (0.36 ratio) and Taiwan (0.27 ratio) heavily rely on TV-Shows, reflecting the global demand for K-dramas and regional series. Conversely, countries like the United Arab Emirates (36:1) and the Philippines (26.67:1) almost exclusively supply feature films.

- **Acquisition Seasonality:** The temporal aggregation pipeline [Figure 2.15] proved that Netflix does not scale its catalog linearly throughout the year. Content loading systematically peaks in December (982 titles), June (858 titles), and September (823 titles). This indicates an operational strategy that prioritizes bulk content loading right before major holiday periods, summer breaks, and the traditional fall

television premiere cycles.

- **Niche Catalog Segmentation:** Foundational queries demonstrated that Netflix maintains highly specific, easily queryable sub-catalogs. For instance, the platform added 496 new TV-Shows in 2021 alone [Figure 2.8], and maintains a solid baseline of highly targeted family-friendly content, with 119 movies strictly rated for kids and clocking in under 60 minutes [Figure 2.9].
- **The Cost of Data Sparsity:** The analysis practically validated the importance of the data wrangling phase. This serves as a reminder that while NoSQL architectures offer immense schema flexibility, data cleanliness and preprocessing remain of paramount importance for accurate business intelligence.

3.2. Final Remarks

The transition to a NoSQL architecture fundamentally addresses the core business challenges faced by an industry leader managing a massive, constantly evolving catalog for approximately 200 million monthly active users. As demonstrated throughout the project, traditional Relational Database Management Systems (DBMS) and their rigid, tabular schemas are inherently not able to handle the growing complexity of entertainment metadata, such as highly variable cast lists, heterogeneous duration metrics, and multi-tag genre categorizations.

By moving away from a traditional relational approach, we have illustrated how data ceases to be merely a static inventory of information and instead becomes a dynamic, strategic asset. The implementation of a flexible, document-oriented schema directly enables the operational efficiency required to ingest and organize complex data without the friction of constant schema migrations. Ultimately, this architectural shift does more than just optimize backend operations; it provides the essential foundation for extracting the strategic insights needed for regional content investments, while powering the rapid data retrieval necessary for optimizing the end-user's search and recommendation experience.